

Out of FLASH memory walkthrough with samples

[BHAudio](#) 1 September 26, 2023, 10:43pm

Hello all,

I wrote this as I was working through my “out of flash memory” options and truly digging in to figure out how to utilize the different memory options on the Daisy Seed. As I was writing this out I thought it would be good to share what I have learned so that it may help other people.

Before I get into it I want to say a huge thank you to all who are on the forums and the Discord channel and a special thank you to [@antisvin](#) and [@shensley](#) for their thoughts on this write-up. Without the collective help and answers I would not be able to complete this write-up, nor get to this next level to build my upcoming sequencer.

Please note that I am new to embedded programming and this is my first deep dive into solving this type of problem. If anyone reads through this and has questions, please ask, and moreover if you see something incorrect or a better way to solve this please let me know.

Overview

This document is set up with sections discussing options on ways to manage the challenge of having a full FLASH. Each section is designed to be a standalone to help you try out the one idea and if that is enough, excellent. If not, you can try the next section.

Here is the high level of the concepts addressed in this article.

1. Reduce code size by turning on the optimizer

This may be good enough to help you keep coding. 😊

2. Putting large memory usage in SDRAM

If you have large memory structures, you can move them to SDRAM to clear up space.

3. Moving to the Daisy Bootloader to help load content into other areas of available RAM helping you have more space for coding. We will explore using the SRAM bootloader option with the Daisy Seed bootloader. Note: there is a QSPI option as well, not covered here.

4. You moved to the SRAM bootloader but your DTCMRAM is full

This will start with an exploration of memory types, an observation about the linker scripts, a discussion on the `DSY_SDRAM_BSS`, more on the linker script sections, and how we can target code to go into different memory segments.

5. Targeting full object files to different memory locations

We will extend the use of different code segments and show how you can place an entire object file code segment into a specific location in memory. This will allow you to take advantage of optimizing the use of memory spaces based on your needs.

Note: these are not the only options to solve this challenge. This just happens to be what I learned and I want to share this if it can help others.

Turn on the optimizer!

If things are too large, can we reduce the size?

The first time I came across this just optimizing the code helped me get enough space to continue coding for some time. Here is one way to do this:

1. Go into the Makefile for your project

2. Add in the line: `OPT = -Os`

Note: I added these flags between the Sources section and the library section.

Now I can compile and keep coding.

A handy debugging tip:

If you are stepping through your code and you want to see a variable that you have put in the code and VS Code notes: “optimized out” you can add `volatile` to the variable declaration and it will not be optimized out.

Note: you would not want to keep a variable as “volatile” once you have completed debugging that section of code because volatile variables have runtime penalty for data access performance and also end up using more memory as they’re not “optimized out” when no longer needed.

Example:

```
uint_32t  MyVariable;
```

Changes to:

```
volatile uint_32t  MyVariable;
```

Putting large memory blocks in SDRAM

If you have large arrays or other memory structures, you can allocate them in the SDRAM. There is a good article on how to do this here called “Getting Started - External SDRAM” (libDaisy: Getting Started - External SDRAM ([electro-smith.github.io](https://forum.electro-smith.com/t/out-of-flash-memory-walkthrough-with-samples/4370/print)))

Note: SDRAM is the slowest memory and should be used only if other sections are not suitable due to size limitations. You may also want to consider other optimizations such as using SDRAM for the larger storage and bring smaller sections into higher performance memory depending on your application needs.

One important note from this article is around initializing and class structure. You will need to have an initialization function that you call after the hardware init function. This is because the SDRAM needs to be

setup for use before anything can be set in that memory location. If your background is from a Desktop programming environment, you may be used to setting variables at compile time. This does not work for external memory on these devices because the hardware initialization needs to be run so that the processor knows about the memory region and can use it correctly. This is why you will need to initialize your classes and variables after hardware initialization.

When code optimization and using SDRAM is no longer enough:

I kept coding and adding features that required new libraries, such as USB MIDI, and suddenly even optimized code was not enough.

Enter the Daisy Bootloader!

After doing a few searches on the forums and asking some questions I was directed to this article: “Getting Started - Daisy Bootloader” (libDaisy: Getting Started - Daisy Bootloader (electro-smith.github.io))

Where to boot from?

Once I started reading through this article I had to decide where I am going to boot from and how best to pick a way to boot during the development phase and what will work best when I ship my device? I decided to start with the SRAM boot loader. I may switch to QSPI, or write my own linker script, in my final version but that will be a write-up for another day. 😊

Setting up the SRAM boot:

There are a few articles on this, and for completeness I added the steps I took to get this working:

There are two main changes to allow this to work:

1. flash the Daisy bootloader to your Daisy Seed
2. Update the launch.json file in VS Code to ensure the debugger can connect and that “run” / F5 will use the proper task to program the device.

Flashing the new Daisy Bootloader:

We need to update the Makefile so that the build process will know to target using the SRAM space when using the Daisy Bootloader.

1. Go into your Makefile for the project.
2. Add in the following: `APP_TYPE = BOOT_SRAM`
This will have the build system use a new linker script that will move where the code goes into memory. This linker script will target loading code into the SRAM and DTCMRAM and this should give more coding space for most projects. We will discuss linker files later in this article if you wish to know a little more.
3. Save the Makefile

4. Go to your bash shell for the project

5. Reset the Daisy Seed to DFU

6. Type in: “make program-boot”

Note: the boot loader will now be loaded into the FLASH memory of the Daisy Seed and this memory will not be available for coding.

7. Reset the Daisy Seed

Notice: the new bootloader will go through a boot cycle of 2.5 seconds wait for a DFU connection and then go into its boot cycle looking for the other places to load code (e.g. SDCard, external USB, etc.). Read the Daisy Bootloader link at the top of this section for more information.

8. Put the Daisy bootloader into DFU mode without cycling through the other modes by:

Press and release reset, then press and release boot button.

Notice: The Daisy Seed LED will be the “breathing” LED indicating the Daisy bootloader is waiting for a DFU connection.

We are getting closer, let’s make sure VS Code can deploy and debug this new setup.

Configure VS Code:

We will be updating the launch.json to change the default F5 / Run → Debug code will execute the proper task and we will also add a debugger command to ensure debugging works properly.

1. Go to the .vscode directory for your project

2. Open the launch.json

3. Go to the line “preLaunchTask”

4. Change the line to: “preLaunchTask”: to be “build_and_program_dfu”,

Note: this is to allow the F5 compile to build and program via DFU.

5. Go to the section for: “openOCDLaunchCommands”

6. Add in the line “gdb_breakpoint_override hard”

Notes:

a. I added this as the last line in this openOCDLaunchCommands section, for me it is right after the “reset init”

b. This is needed to ensure the debugger can connect

Code and debug:

With the above completed you should be able to code, debug, and keep working away.

There is a downside that I have not quite gotten around yet, and that is, sometimes I still have to reach the buttons to do the DFU setup. I've looked at other ways to do a software reset of the Daisy seed so that I don't have to reach the buttons as often, but this is still a work in progress for me. Likely another article in the future. 😊

Note: there is an oddity that shows up for me when I first start debugging after I load up a new session. I reset the Daisy Bootloader bit the press and release reset and then press and release boot. The first time I go to debug the debugger takes a while to start up. During this time the Daisy ends up in the Bootloader looking for connections but is not running the code. I simply restart the deployment / debugging and this causes the project to rebuild, deploy via DFU, and then everything connects to and is ready to run and debug live.

DTCMRAM is full!

After coding for a while you are now getting the DTCMRAM is full. When you get to this state, here are two options I chose to explore:

1. See the section above about using the SDRAM for larger arrays and memory allocations.
2. Using a custom linker script to locate parts of your code in different memory areas.

Being a noob to programming embedded systems this was a unique learning journey and this is the core of why I wrote this article.

A note as you read further through this document:

From here, this is where I only know enough knowledge to have solved my specific issue. This did work for me, and may help you as you go forward. Please note I am sure there are more options beyond this that may work well for you.

Below I have added a few references here that helped me get to this point and I want to share these with you as well so you can dig in more if you wish.

References

These are the core articles I used to create the rest this document that follows and I want to make them available to you for your own learning.

- [Mastering the GNU linker script - AllThingsEmbedded](#)
- Bare metal embedded lecture series on YouTube: [Bare metal embedded lecture-4: Writing linker scripts and section placement - YouTube](#)

Note: this is a great series, and I went through a few of the lectures, this is the one that helped me most along with the two other items noted below.

- Using Linker Scripts: [Using LD, the GNU linker - Linker Scripts \(colorado.edu\)](#)
- Reading the Daisy linker scripts [libDaisy/core at master · electro-smith/libDaisy \(github.com\)](#)

So many memory types!

As I dug in I started looking more closely at all of the memory types, their tradeoffs, and how the compiler ... well linker ... decides to put what things in what area of memory. What follows here is my observations how this all goes together. If you want to jump to a summary of what I did for my code feel free to go to the section “My Custom Linker Scripts” at the end of the article.

Compiler output

After a compile I always noticed there is the section saying how much of each memory type is used. I never really thought much of it until the day I got the FLASH is full and started reading through options on how to solve these issues.

Memory region	Used Size	Region Size	%age Used
FLASH:	0 GB	128 KB	0.00%
DTCMRAM:	7440 B	128 KB	5.68%
SRAM:	46000 B	512 KB	8.77%
RAM_D2_DMA:	16 KB	32 KB	50.00%
RAM_D2:	0 GB	256 KB	0.00%
RAM_D3:	0 GB	64 KB	0.00%
ITCMRAM:	0 GB	64 KB	0.00%
SDRAM:	0 GB	64 MB	0.00%
QSPIFLASH:	0 GB	7936 KB	0.00%

(this sample comes from the Daisy Bootloader article)

As I was digging through articles and reading about the bootloader, I came across the discussion on the linker files and decided to dig into the linker file where it was showing the memory sizes and locations.

Linker File MEMORY section

When working on using the Daisy Bootloader I found my way to the STM32H750IB_sram.lds [libDaisy/core/STM32H750IB_sram.lds at master · electro-smith/libDaisy \(github.com\)](https://github.com/libDaisy/core/STM32H750IB_sram.lds)

When looking through the file I noticed the MEMORY section at the top of the file and realized this correlates to the output of the build systems memory usage data.

```
MEMORY
{
    FLASH (RX)      : ORIGIN = 0x08000000, LENGTH = 128K
    DTCMRAM (RWX)   : ORIGIN = 0x20000000, LENGTH = 128K
    SRAM (RWX)      : ORIGIN = 0x24000000, LENGTH = 512K - 32K
    RAM_D2_DMA (RWX) : ORIGIN = 0x30000000, LENGTH = 32K
    RAM_D2 (RWX)    : ORIGIN = 0x30008000, LENGTH = 256K
    RAM_D3 (RWX)    : ORIGIN = 0x38000000, LENGTH = 64K
    ITCMRAM (RWX)   : ORIGIN = 0x00000000, LENGTH = 64K
    SDRAM (RWX)     : ORIGIN = 0xc0000000, LENGTH = 64M
```

```
QSPIFLASH (RX): ORIGIN = 0x90040000, LENGTH = 7936K
}
```

“.sdrām_bss” linker file section and the DSY_SDRAM_BSS macro:

As I kept digging deeper into the LDS file I noticed this section:

```
.sdrām_bss (NOLOAD) :
{
    . = ALIGN(4);
    _ssdrām_bss = .;

    PROVIDE(__sdrām_bss_start = _ssdrām_bss);
    *(.sdrām_bss)
    *(.sdrām_bss*)
    . = ALIGN(4);
    _esdrām_bss = .;

    PROVIDE(__sdrām_bss_end = _esdrām_bss);
} > SDRAM
```

I recalled that the “.sdrām_bss” section showed up in the macro discussion from the Getting Started - External SDRAM libDaisy: [libDaisy: Getting Started - External SDRAM \(electro-smith.github.io\)](https://github.com/electro-smith/libDaisy/blob/master/Getting%20Started%20-%20External%20SDRAM.md)

In this article it shows using the macro:

```
float __attribute__((section(“.sdrām_bss"))) my_buffer[1024];
```

The article notes that they created a macro called DSY_SDRAM_BSS and if you follow the link it turns out there are two macros for the SDRAM section. One for SDRAM_BSS and one for SDRAM_DATA.

This lead me deeper into wanting to better understand: What are the BSS, vs DATA, vs other sections such as .TEXT that are in the linker file. I ended up digging around and finding the article Mastering the GNU linker script - AllThingsEmbedded. Let’s dig into a high level view of the .bss and other sections.

.bss, .data, and other sections in the linker

Digging more into the Mastering the GNU linker script - [AllThingsEmbedded](https://www.allthingsembedded.com/post/mastering-the-gnu-linker-script/) it did a great job of talking through what the different sections are. Here is a direct quote from the article on the different sections:

.text: This section contains the code. This is, the machine language instructions that will be executed by the processor. In here we will find symbols that reference the functions in your object file.

.rodata: This contains any data that is marked as read only. It is not unusual to find this data interleaved with the text section.

.data: This section contains initialized global and static variables. Any global object that has been explicitly initialized to a value different than zero.

.bss: Contains all uninitialized global and static variables. These are usually zeroed out by the startup code before we reach the main function. However, in an embedded system we usually provide our own startup code, which means we need to remember to do this ourselves. I wrote a nice article about the startup code a while back here.

Putting the memory sections and the Daisy memory macros together:

Given the info above with the SRAM.LDS linker file we can follow the flow from your C/C++ code through to what the linker does and where your type of data will land in the given memory space.

If I have an array: I can now direct that array to SDRAM via `DSY_SDRAM_BSS`, run the build data and see that the array will now land in the SDRAM section of the build output.

Note: the BSS section is all about uninitialized memory. You will need to initialize your memory after the hardware initialization function is called because the external memory is not available to the processor until hardware initialization is complete. This is discussed in the article: [libDaisy: Getting Started - External SDRAM \(electro-smith.github.io\)](https://electro-smith.github.io/libDaisy/GettingStarted-ExternalSDRAM/)

Now that I understand there is a direct causation of the use of the macro adding the attributes to the object file and that those attributes help direct the linker to place the contents in a specific memory location I now wanted to see if I can redirect code from the DTCMRAM to the SRAM section.

Putting code into SRAM over DTCMRAM since there is more SRAM:

Note on my specific results: once I added some extra libraries my build was failing due to my DTCMRAM being at 120.02% full, and that won't work. 😊 When I completed the work below my build showed DTCMRAM usage is at 1.42% and SRAM usage is now at 55.08%. We can build and keep coding!

Let's walk through how to get some the full DTCMRAM redirected to SRAM.

After listening through this video: [\(54\) Bare metal embedded lecture-4: Writing linker scripts and section placement - YouTube](#) he talks about how the section targets memory by the use of the “} > MEMORY” at the end of each section in the linker file.

In looking more closely at the `_SRAM.LDS` linker script I noticed this section “.bss“

```
.bss (NOLOAD) :
{
    . = ALIGN(4);
    _sbss = .;

    PROVIDE(__bss_start__ = _sbss);
    *(.bss)
    *(.bss*)
    *(COMMON)
    . = ALIGN(4);
    _ebss = .;
```



```
        PROVIDE(__bss_end__ = _ebss);
    } > DTCMRAM
```

Noticing the memory at the end of the section where it shows “} > DTCMRAM” I decided to change that from DTCMRAM to SRAM. Now my .bss section looks like this:

```
.bss (NOLOAD) :
{
    . = ALIGN(4);
    _sbss = .;

    PROVIDE(__bss_start__ = _sbss);
    *(.bss)
    *(.bss*)
    *(COMMON)
    . = ALIGN(4);
    _ebss = .;

    PROVIDE(__bss_end__ = _ebss);
} > SRAM
```

This worked great as now my uninitialized code is now running in SRAM and I have plenty of space in DTCMRAM and used a little over half of my SRAM.

Sweet, I’m up and coding again with plenty of space!

Note: the full details of the changes are below in the “custom linker scripts” section...

Tradeoffs for memory types:

It is important to know that there are specific tradeoffs in the type of memory one uses on these devices. In the stm32h750ib.pdf from STM32H750IB - High-performance and DSP with DP-FPU, Arm Cortex-M7 MCU with 128Kbytes of Flash memory, 1MB RAM, 480 MHz CPU, L1 cache, external memory interface, JPEG codec, HW crypto, large set of peripherals - STMicroelectronics

From Page 24, section 3.3.3 Embedded SRAM

“RAM mapped to TCM interface (ITCM and DTCM): Both ITCM and DTCM RAMs are 0 wait state”

ITCM and DTCM RAMS have very fast direct connections to the CPU whereas some of the other memories onboard the chip will be running slower than the CPU requiring wait states to fetch and store memory.

For me, I would like to ensure that my core processing of my main sequencer is running as fast as possible. This brought me to my next question: Can I target code files to go into specific parts of the system memory. It turns out the answer is yes. Read on if you wish to know more. 😊

A few notes on DMA buffers and the SDMMC memory usage:

DMA user-provided buffers must be located in memory outside of the TCM memory.

SDMMC requires that any objects that interact with the peripheral (FIL, buffers, etc.) are located in the AXI SRAM (or the SDRAM).

Targeting where files of code can go in memory:

At this point I was wondering: Can I now target where specific parts of my code go into memory given what I have learned above?

After reading through this article I found: [Using LD, the GNU linker - Linker Scripts \(colorado.edu\)](https://colorado.edu/linker/scripts) this helped me better understand what was possible in a linker script. One area that stuck out for me were two sections in this article under the “Input Section Descriptions”:

First part I noticed:

You can specify a file name to include sections from a particular file. You would do this if one or more of your files contain special data that needs to be at a particular location in memory. For example:

```
data.o(.data)
```

If you use a file name without a list of sections, then all sections in the input file will be included in the output section. This is not commonly done, but it may be useful on occasion.

Later in that section:

If a file name matches more than one wildcard pattern, or if a file name appears explicitly and is also matched by a wildcard pattern, the linker will use the first match in the linker script. For example, this sequence of input section descriptions is probably in error, because the `data.o` rule will not be used:

```
.data : { *(.data) }  
.data1 : { data.o(.data) }
```

I was able to use this technique to target my core sequencer object file (seq.o) to run in the DTCMRAM. I did this by creating a new section towards the top of linker script to ensure the seq.o was prioritized to be put into the DTCMRAM early in the linker processing.

I cover the details below in my “custom linker scripts” section.

My Custom linker scripts:

There were two custom linker scripts I used to help solve my out of memory for the DTCMRAM based on the _SRAM.lds file.

Moving uninitialized data from DTCMRAM to SRAM:

This is discussed in the section above: “Putting code into SRAM over DTCMRAM since there is more SRAM.” Here are the specific steps I added in my project.

1. Copy the file libDaisy/core/STM32H750IB_sram.lds to your project directory

Note: I renamed mine to reduce any confusion on what .lds file I was editing and/or looking at.

I'll show copied file as STM32H750IB_my_sram.lds for this part of the document.

2. Open STM32H750IB_my_sram.lds in VS Code
3. Find the section that start with: ".bss (NOLOAD) :"
4. Go to the bottom of the section: "{} > DTCMRAM"
5. Change that section to read: "{} > SRAM" (no quotes)
6. Save STM32H750IB_my_sram.lds
7. Open your Makefile
8. Add in the line LDSCRIPT = ./STM32H750IB_my_sram.lds
9. Save Makefile
10. Compile

Note: It is critical to know that this will move only your uninitialized data to SRAM.

In my case this is a large section of my code given I have a large amount of data to manage the 16 CV outs for the sequencer.

My project output before the change:

Notice below that the DTCMRAM is at 120.02% full – that won't work. 😊

Memory region	Used Size	Region Size	%age Used
FLASH:	0 GB	128 KB	0.00%
DTCMRAM:	157312 B	128 KB	120.02%
SRAM:	115288 B	480 KB	23.46%
RAM_D2_DMA:	17200 B	32 KB	52.49%
RAM_D2:	0 GB	256 KB	0.00%
RAM_D3:	0 GB	64 KB	0.00%
ITCMRAM:	0 GB	64 KB	0.00%
SDRAM:	157140 B	64 MB	0.23%
QSPIFLASH:	157540 B	7 MB	2.15%

My project output after the change:

Notice below that: DTCMRAM usage is at 1.42% and SRAM usage is now at 55.08%

Memory region	Used Size	Region Size	%age Used
FLASH:	0 GB	128 KB	0.00%
DTCMRAM:	1856 B	128 KB	1.42%
SRAM:	270744 B	480 KB	55.08%
RAM_D2_DMA:	17200 B	32 KB	52.49%
RAM_D2:	0 GB	256 KB	0.00%
RAM_D3:	0 GB	64 KB	0.00%
ITCMRAM:	0 GB	64 KB	0.00%
SDRAM:	157140 B	64 MB	0.23%
QSPIFLASH:	157540 B	7 MB	2.15%

Sweet, We can run code and debug! 😊

Targeting my SEQ.O object files to DTCMRAM:

Given my sequencer runs in the audio callback I want to target having all the core memory access for the sequencer to be in DTCMRAM so that it all runs in memory with no wait states. (see section above on “Tradeoffs of memory types”).

The result for me is that DTCMRAM usage started at 1.42% usage based on our previous change. Post the following change the DTCMRAM ended up at 68.99% usage and SRAM ended up at 37.06%! now my core data structures for my sequencers run at zero wait states to the CPU.

I did this by adding the following section in STM32H750IB_my_sram.ldsI from above.

1. Open the STM32H750IB_my_sram.lds
2. Go to the line: “_sidata = LOADADDR(.data);”
3. Add in a new section after the above line.

Note: I put this towards the top of the file to ensure this processed first and that the seq.o.bss / .bss* functions target the DTCMRAM. (see the section above “Targeting where files of code can go in memory:” on why I put this towards the top)

Here is my example:

```
.dtcmram_bss_seq (NOLOAD) :
{
    . = ALIGN(4);
    dtcmram_bss_seq = .;

    PROVIDE(__dtcmram_bss_seq_start__ = dtcmram_bss_seq);
    build/[your_file1.o (.bss)
    build/[your_file1.o] (.bss*)
    . = ALIGN(4);
    _edtcram_bss_seq = .;

    PROVIDE(__dtcmram_bss_end_seq__ = _edtcram_bss_seq);
} > DTCMRAM
```

4. Adjust the file you wish to have linked directly to what matches your project.
5. I would suggest updating the *start* and *end* to better match your project.
6. Save the file
7. Go to the top of the file in the “MEMORY” section
8. Change the line that has: “QSPIFLASH (RX): ORIGIN = 0x90040000, LENGTH = xxxk” to: “QSPIFLASH (RX): ORIGIN = 0x90040000, LENGTH = 7M” (no quotes)

Note: this is due to a DFU load issue when using the current Daisy Bootloader. This change is a fix that works for me, and this should not be needed when the bootloader is updated.

9. Compile

Enjoy your ability to target code into different areas of memory to help you optimize your code for your project.

My project output before the change:

Notice the DTCMRAM usage is at 1.42% usage based on our previous change.

Memory region	Used Size	Region Size	%age Used
FLASH:	0 GB	128 KB	0.00%
DTCMRAM:	1856 B	128 KB	1.42%
SRAM:	270744 B	480 KB	55.08%
RAM_D2_DMA:	17200 B	32 KB	52.49%
RAM_D2:	0 GB	256 KB	0.00%
RAM_D3:	0 GB	64 KB	0.00%
ITCMRAM:	0 GB	64 KB	0.00%
SDRAM:	157140 B	64 MB	0.23%
QSPIFLASH:	157540 B	7 MB	2.15%

Let's see what the change did for us.

My project output after the change:

Notice below that the DTCMRAM is now at 68.99% usage and SRAM is now at 37.06%!

Memory region	Used Size	Region Size	%age Used
FLASH:	0 GB	128 KB	0.00%
DTCMRAM:	90424 B	128 KB	68.99%
SRAM:	182176 B	480 KB	37.06%
RAM_D2_DMA:	17200 B	32 KB	52.49%
RAM_D2:	0 GB	256 KB	0.00%
RAM_D3:	0 GB	64 KB	0.00%
ITCMRAM:	0 GB	64 KB	0.00%
SDRAM:	157140 B	64 MB	0.23%
QSPIFLASH:	157540 B	7 MB	2.15%

We now have the core sequencer .bss / uninitialized variables memory running in DTCMRAM at zero wait states to the CPU.

Can I use ITCMRAM?

I have not personally ventured into using ITCMRAM for my project but there are a few notes that were shared with me on this topic:

- Currently the Daisy bootloader will not copy data to that memory region.
- Another gotcha with ITCMRAM is that it's address starts with 0, so a pointer to function that is stored first on it will end up indistinguishable from nullptr 😬 It's best to just admit first word from that section

in your linker script to avoid bugs later on.

Future work:

Needed changes:

- I wish to detail how I did the work for the custom linker script so that I am not editing the main Daisy Seed `_SRAM.LDS` file.
- Look at bringing the `.text` section of the `seq.o` into the DTCMRAM
- Look into the use of the ITCMRAM and do a follow up article if I end up needing to use this space. Considering a custom linker script without the Daisy bootloader

On my backlog of work is to consider not using the Daisy Bootloader but rather build a custom linker script and bootloader. When I do get to this I will write a follow-up tutorial.

Closing thoughts

I hope you find this walkthrough useful as I wanted to try to collect what I have learned in one place and share it with this community.

Happy coding to you!

19 Likes

[Reading a table with DaisySP](#)

[Additional Storage For Larger Patches](#)

[Help storing data in QSPIFLASH - dfu-util is not writeable error](#)

[Performance tips](#)

[cricketbee](#) 2 September 27, 2023, 4:56am

thanks so much for putting this together! printing this out as I know it will be a great resource.

[antisvin](#) 3 September 27, 2023, 12:02pm

Congratulations on finishing this epic post 😊 Actually I'm not sure if it is fully finished as it covers a lot of topics at once.

BHAudio:

SDMMC requires that any objects that interact with the peripheral

While it is certainly true that SDMMC DMA can access only specific memory regions, it's also possible to make a workaround that allows using arbitrary destination with SDRAM while still using DMA transfers. The idea is to have only a small buffer for a single transaction on AXI SRAM and perform another copy to final destination buffer from SDMMC callback.

IIRC, CubeMX generates this sort of code and adapted version of it worked well for adding SD card support on Owlsey. This has some overhead as we have to perform an extra copy between different areas of memory, but it was necessary when memory gets allocated dynamically and can end up in any region.

BHAudio:

there are two macros for the SDRAM section. One for SDRAM_BSS and one for SDRAM_DATA.

Note that you have to [modify startup file](#) in order to get an extra `.data` section on SDRAM. And in order for it to work, SDRAM must be initialized before your firmware runs. This means that it can only work if you run bootloader first, otherwise SDRAM will be initialized by firmware at later time.

BHAudio:

I would like to ensure that my core processing of my main sequencer is running as fast as possible.

Your core processing code will likely be running as fast as possible from any area thanks to MCU cache! Only parts of it that are less frequently accessed will gain benefits from running from faster memories. And even for those cases MCU would prefetch your code in advance, reducing amount of wait time to some extent. My guess would be that the best use for *TCMRAM is storing most frequently called DMA callbacks.

1 Like

[BHAudio](#) 4 September 27, 2023, 8:43pm

[@antisvin](#) ,

Thank you for the feedback, I'll look to update the primary document based on the feedback. I'll note the updates based on your feedback once I complete them, so that way when people read through it, they have the all the updates. 😊

And yes, good point on quite being finished, but it sure was a huge help to me to write it out so it was reasonably organized for me to put the techniques to use.

As a note: Once I get my device out I plan to OpenSource the software so I can come back in here and in better code pointers to a working project.

Thank you again for the feedback and comments. More to come. 😊

4 Likes

[BHAudio](#) 5 October 7, 2023, 7:21pm

[@antisvin](#) ,

Do you have a sample set of code we could add in here and/or a pointer to an example?

antisvin:

While it is certainly true that SDMMC DMA can access only specific memory regions, it's also possible to make a workaround that allows using arbitrary destination with SDRAM while still using DMA transfers. The idea is to have only a small buffer for a single transaction on AXI SRAM and perform another copy to final destination buffer from SDMMC callback.

IIRC, CubeMX generates this sort of code and adapted version of it worked well for adding SD card support on Owlsy. This has some overhead as we have to perform an extra copy between different areas of memory, but it was necessary when memory gets allocated dynamically and can end up in any region.

[antisvin](#) 6 October 8, 2023, 9:57am

I don't have an example that uses libDaisy specifically and it's [a bit different](#), but the general idea is to add a scratch buffer:

```
static uint8_t scratch[BLOCKSIZE];
```


That buffer must be allocated in AXI SRAM since it's uses DMA transfers. Caching should probably not be used as writes to it are done by DMA, otherwise it should be aligned to cache line width and invalidated on each transfer.

Then your [SD read function](#) will do something like:

```
while(sd_state == MSD_OK && count--) {
    if(BSP_SD_ReadBlocks_DMA(
        (uint32_t *)buff, sector, 1)
        != HAL_OK) {
        sd_state = MSD_ERROR;
    }
    else {
        // poll for transfer complete flag, etc..
        // copy to final buffer after successful transfer and cache invalidation
        memcpy(buff, scratch, BLOCKSIZE);
        buff += BLOCKSIZE;
        sector++;
    }
}
```

This would have some overhead compared to a single DMA transaction, but I imagine it would still be faster than using SD IO without DMA.

[mjkirk12](#) 7 October 15, 2023, 4:32am

Per the datasheet, the Daisy CPU has 128K flash. Should this be sufficient for most audio applications?

I see there are pin compatible parts in same package with 2 MB Flash (16 X larger).

Would the Daisy team consider offering versions (at higher price) ?

For reference:

[STM32H750IBK6](#) current part for Daisy Seed 128K Flash with Crypto

[STM32H753IHK6](#) 2MB flash with Crypto

[STM32H743IHK6](#) 2MB flash without Crypto

[tunagenes](#) 8 October 15, 2023, 4:59am

I think that's a 'good' question, but wanted to check that you noticed the Daisy Seed has an 8MB 'External' Flash on the board. It would be nice to have larger on chip flash, and it has a few advantages

besides size.

[mjkirk12](#) 9 October 15, 2023, 12:40pm

Can the STM32H processor run code directly from this external QSPI flash?

For a 32 bit ARM, this means 8 reads are needed to get the instruction word. At 480 MHz, the effective execution rate of QSPI would be 480/8 or 60 MHz. Could be even slower - I don't have experience with running code from QSPI flash. Unless there is a bootloader that can move this code into RAM and execute from RAM. That would speed things up.

[BHAudio](#) 10 November 8, 2023, 12:55am

A few things here:

1. I found that the SD Card streams audio well with the DMA buffer
2. I have a modified version of the: <https://github.com/electro-smith/DaisyExamples/tree/master/seed/WavPlayer> that does a fine job of using a small part of SRAM but this could easily work from the SDRAM as well.
3. for my project I'm using the QSPI flash as persistent storage for save / loads and soon some user preferences. That way, the core of the device (a sequencer) always be saved.

In short the 128K FLASH was a challenge but I was able to work around it and use a combo of the SD card, QSPI Flash, and move forward from what is there today.

Hope that helps a little.

2 Likes

[BHAudio](#) 11 March 11, 2024, 2:20pm

SDMMC and BOOT_SRAM

I wanted to follow up on the SDMMC and BOOT_SRAM setup I ended up using here just in case others have questions and stop here to read through this tutorial.

I used the technique noted [here](#)

```
#define DSY_TEXT __attribute__((section(".text")))
static DSY_TEXT SdmmcHandler    sdcard;
static DSY_TEXT FatFSInterface fsi;
static DSY_TEXT AudioDesc      audioDesc;
```

Notes:

1. The AudioDesc is a class setup that manages all of the file IO for reading out audio descriptions of what is happening on my sequencer. I wanted to ensure that this is also running out of the SRAM space because this contains all of the file (FIL) that cannot be in DTCMRAM.
2. File system interface also needs to be in the SRAM space and not DTCMRAM
3. I chose to also put the Sdmmc handler there as well, but I do not believe this is required but it works having it there as well.

For reference: here is my linker script section:

```
.text :
{
    . = ALIGN(4);
    _stext = .;

    *(.text)
    *(.text*)
    *(.rodata)
    *(.rodata*)
    *(.glue_7)
    *(.glue_7t)
    KEEP(*(.init))
    KEEP(*(.fini))
    . = ALIGN(4);
    _etext = .;
} > SRAM
```

With all of the above I am able to use the BOOT_SRAM config and still have all of the files stream for the SD Card and work as expected.

Hope this helps give you another option on ensuring the SDMMC works when in the BOOT_SRAM config.

3 Likes

[tunagenes](#) 12 March 11, 2024, 4:24pm

Thanks [@BHAudio](#) for the doc and the pointer!

[horstmaista](#) 13 March 29, 2024, 4:54am

Yeah! Thanks [@BHAudio](#) this just finished a long tour of digging around!
This should be part of an example file!

[Ali](#) 14 July 7, 2024, 11:11pm

Thanks for this Writeup!

[BMP4](#) 15 October 25, 2024, 7:57pm

Thank you so much for this amazing guide! As I was following along I made myself a TL;DR version for moving the code from FLASH to SRAM, which might be useful for others. It should really just work, but let me know if it doesn't and I'll edit it accordingly!

Prepping VS Code for running and debugging

in `.vscode/launch.json` :

- change `"preLaunchTask": "build_all_debug"`, to `"preLaunchTask": "build_and_program_dfu"`,
- in `openOCDLaunchCommands`, add `"gdb_breakpoint_override hard"` as the last step

Add a custom linker script to move code from DTCMRAM to SRAM

The custom linker script is optional if you don't need to move code out of DTCMRAM at the moment, but IMO better to set this up for later while you're doing the rest of these instructions

- make a copy of the `submodules/libDaisy/core/STM32H750IB_sram.lds` linker script at the root of your repo
- in that copy, at the bottom of the `.bss (NOLOAD) :` section, change `} > DTCMRAM` to `} > SRAM`

Edit your Makefile

Add these 2 lines between your `CPP_SOURCES` and `LIBDAISY_DIR` commands:

```
APP_TYPE = BOOT_SRAM
LDSCRIPT = ./STM32H750IB_sram.lds # double-check the location and name of your c
```

Update the bootloader on the DaisyPod

- put the daisy into DFU mode by holding BOOT and pressing RESET button
- in your terminal, at the top of your repo, type `make program-boot` to load the daisy bootloader in flash

Debug in VS Code

- put daisy seed in DFU mode: Press and release RESET, then press and release BOOT
- then F5 from VS Code should *just work* TM

And that's it! From now on whenever you want to debug/program your seed you'll have to preset RESET, then BOOT

1 Like

[Should I use Zadig to update the Daisy Bootloader driver?](#)

[BHAudio](#) 16 December 28, 2024, 6:36pm

[@BMP4](#) , great summary - thanks. 😊

1 Like

[wailem](#) 17 May 29, 2025, 9:44pm

Did anyone come across an issue like this? [No DFU device detected after bootloader installed](#). I'm basically stuck between "Flashing the new Daisy Bootloader:" and "Configure VS Code:" steps. I assume running `build_and_program-dfu` should work without the "Configure VS Code" steps.

EDIT: For anyone arriving here, the answer to this problem in is that thread!

[Uzo](#) 18 January 29, 2026, 5:54pm

Thanks so much for taking the time to do this write up and including the resources. Super useful! Not quite at the custom linker stage yet but I know it's only a question of time so will be coming back to reference this soon